WHEN DOES ITERATIVE SIDELOBE NULLING STOP? TERMINATION OF A PERTURBATIVE BEAMFORMING METHOD, WITH A MECHANIZED PROOF IN ISABELLE/HOL

DUSTIN BRYANT

Abstract. The perturbative method shapes the radiation pattern of a phased array by repeatedly placing a null on its loudest sidelobe. Starting from the weights that steer the beam to a focus direction, each iteration evaluates the pattern on a grid of directions, finds its strict local maxima, sets the main lobe aside, and subtracts from the weight vector the smallest perturbation that cancels the pattern at the loudest remaining peak above -20 dB. The loop stops when no such peak remains. Whether it stops is not obvious: a null in one direction changes the pattern in every other direction. We show that the answer depends on the array. For two nine-element arrays, a "double cross" and a regular octagon, the method stops after at most 20 and at most 10 corrections, for every focus angle and every rule for breaking ties, with a 30° guard band or without one. Stopping does not mean that every sidelobe is below -20 dB: the test sees only the sampled pattern, and in simulation, at isolated focus angles, the method stops with a sidelobe of up to -5.2 dB that the test misses. For a 32-element circular array of radius four wavelengths the stopping condition cannot be met at all for 20° and 30° bands. We also prove a total-correctness Hoare triple for the method written as a program. All results are proved in Isabelle/HOL. The termination proof follows the pattern of Tucker's computer-assisted proof of the Lorenz attractor: an exact algebraic reduction to finitely many statements about explicit functions on small intervals, which are then settled by interval arithmetic with adaptive subdivision. The difference is that the interval checker is a function defined and proved sound inside the proof assistant. We give the main proofs twice: in ordinary mathematical prose, and as the Isabelle text that was checked.

1. Introduction

A phased array steers a beam electronically: given the positions of its elements and a desired focus direction, one chooses complex weights so that the far-field pattern of the array has its maximum in that direction [1]. The pattern also has secondary maxima, *sidelobes*, which waste power and leak signal in unintended directions. One simple family of methods reduces them by perturbation [18, 6, 14]: starting from the steering weights, find the loudest sidelobe and subtract from the weight vector the smallest perturbation that puts a null there; repeat. The variant studied here, used in the University of Houston drone-array project, evaluates the pattern on a grid of 1000 directions and stops when every strict local maximum of these 1000 samples outside a guard band around the focus, other than the main lobe, is at least 20 dB below the largest sample.

There is no evident reason why this loop should stop. A null placed in one direction changes the pattern in every other direction: it can push other sidelobes above the threshold, and it lowers the main lobe as well, so the threshold, which is relative to the current maximum, moves. Implementations therefore cap the number of iterations. We ask whether the cap is ever needed: for a given array, does the loop stop on its own for every focus direction, and after how many corrections?

Date: November 5, 2026.

array	elements	geometry	band	result
double cross	9	centre; 4 on the axes at 42.21 mm; 4 on the diagonals at 55.40 mm	30°, none	stops within 20 corrections (Theorem 1)
octagon	9	centre; 8 on a circle of radius 48.80 mm	30°, none	stops within 10 corrections (Theorem 2)
circular	32	equally spaced on a circle of radius 400 mm = 4λ	20°, 30°	stopping condition unsatisfiable (Theorem 3)

TABLE 1. The arrays and what is proved about them. The wavelength is $\lambda = 100$ mm (3 GHz).

Results. The answer depends on the array (Table 1, Fig. 1).

- **Theorem 1.** For the nine-element *double cross*, the method stops after at most 20 corrections, for every real focus angle, with a 30° guard band or without one, and for every rule for breaking ties.
- **Theorem 2.** For the regular *octagon*, eight elements on a circle of radius 48.8 mm around a centre element, the same holds with at most 10 corrections.
- **Theorem 3.** For a 32-element uniform circular array of radius 4λ, no choice of weights meets the condition "every direction outside a 20° (or 30°) band is at least 20 dB below the maximum": some direction outside the band always stays above about -15.8 dB (respectively -17.8 dB). A method that stops only when that condition holds therefore runs forever on this array.
- **Theorem 4.** The method, written as a while-program, satisfies a total-correctness Hoare triple: it halts, and on halting every strict peak of the sampled pattern outside the band, other than the main lobe, is at or below -20 dB. The number of corrections is bounded by any bound D for which the run is known to stop; for the double cross $D = 20$, and for the octagon $D = 10$.

What stopping means. The theorems hold for every real focus angle, but the test looks only at the 1000 samples, and there only at strict peaks. So they say nothing about the pattern between samples. In simulation (Section 3.4) the stopped pattern is above -20 dB at isolated focus angles: when the top of a sidelobe falls on two samples of equal value, the test does not see that sidelobe at all.

Method of proof. Theorem 1 is a statement about uncountably many runs (one for each focus angle and each tie rule), and each run is a non-smooth process: which sidelobe is corrected next depends discontinuously on the current pattern. Our proof follows the pattern of Tucker's computer-assisted proof that the Lorenz attractor exists [19, 20], and of Hales' proof of the Kepler conjecture [9]: an exact argument reduces the theorem to finitely many explicit inequalities over compact sets, and these are verified with interval arithmetic and adaptive subdivision [13, 21]. Here the reduction is algebraic. After any fixed sequence of corrections the pattern is a fixed linear function of the initial steering weights (Lemma 2.4), so the runs for all focus angles in a small interval can be followed together, as paths in a finite tree of possible correction sequences.

Unlike the original proofs of Tucker and Hales, our interval checker is not trusted code. It is a function defined in higher-order logic and proved sound in Isabelle/HOL [16, 15], and it is then executed by Isabelle's code generator [7]. This is the approach of Immler's verified ODE solver, which formally re-checked the computational part of Tucker's proof [12], and of the

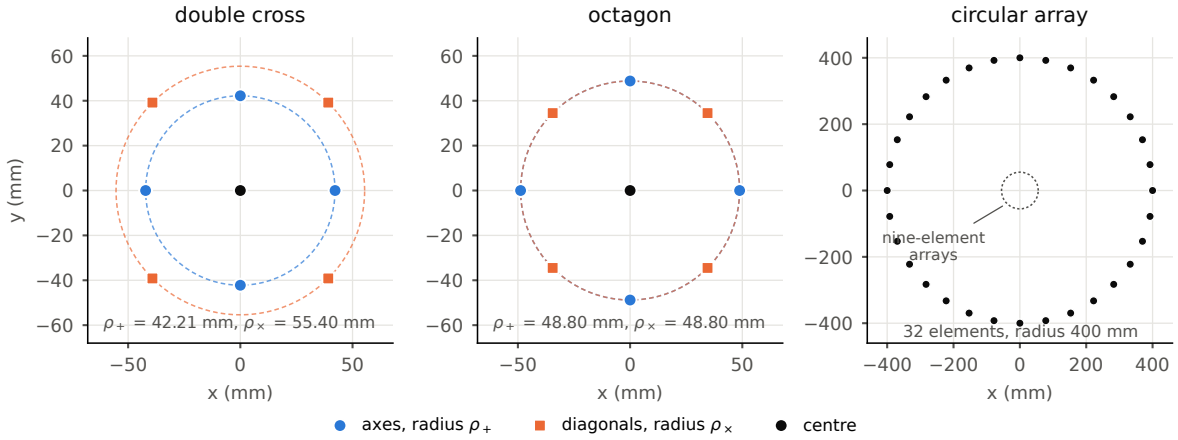


FIGURE 1. Element layouts. Left: the double cross. Middle: the regular octagon (Theorem 2). Right: the circular array, drawn at a smaller scale; the dotted circle shows the size of the nine-element arrays.

verified nonlinear inequalities of the Flyspeck project [17, 8]. Theorem 3 rests on a certificate from linear programming whose arithmetic is also checked by Isabelle. Theorem 4 uses the interaction-tree semantics of Isabelle/UTP [22, 4, 5] and its verification-condition generator, as in our earlier work on numerical methods [2].

What is trusted. The Isabelle kernel, and, for Theorems 1 to 3, the code generator and the ML compiler that evaluate the checker. Isabelle records this dependence: the oracle set of these theorems is exactly {`Code_Generator.holds`, `Code_Generator.holds_by_evaluation`}. The general Hoare triple of Section 7 depends on no oracle. All results concern exact real arithmetic; Section 3.5 explains how a floating-point implementation differs.

How to read this paper. Each definition and theorem is stated in ordinary notation and then shown as Isabelle text. The listings are copied verbatim from the theory files that Isabelle checked; a reader who does not read Isabelle can skip all of them. Section 2 defines the method, Section 3 discusses its stopping condition, Section 4 states the results, Sections 5 to 7 prove them, and Sections 8 and 9 discuss related work and what is not proved.

2. The method

2.1. **Arrays, patterns and the grid.** An array has N isotropic elements at positions $p_j = (x_j, y_j) \in \mathbb{R}^2$, $j = 0, \ldots, N-1$, and radiates at wavelength λ; in all our examples $\lambda = 0.1$ m (3 GHz), so the wavenumber is $k = 2\pi/\lambda = 20\pi$ m^{-1}. In the far field, the response in the azimuth φ to the weights $w \in \mathbb{C}^N$ is the *array factor*

$$F_w(\varphi) = \sum_{j=0}^{N-1} a_j(\varphi)\, w_j, \qquad a_j(\varphi) = e^{\mathrm{i}k(x_j \cos\varphi + y_j \sin\varphi)}. \tag{1}$$

The vector $a(\varphi)$ is the *steering vector*. The method looks at F only on the grid $\varphi_m = 2\pi m/M$, $m \in \mathbb{Z}_M = \{0, \ldots, M-1\}$, with $M = 1000$ (spacing 0.36°), through the powers $P_w(m) = |F_w(\varphi_m)|^2$. Grid indices are taken modulo M, so $m-1$ and $m+1$ are the neighbours of m on the circle.

The formal versions (Listing 1) use the names `steer`, `patt` and `sq` for a, F and P. An array is a function `p` from element indices to positions together with its element count `N`; weights are written `c`. The functions `cis` and `cnj` are $\theta \mapsto e^{\mathrm{i}\theta}$ and complex conjugation.

```
definition kw :: real where "kw = 20 * pi"

definition M :: nat where "M = 1000"

definition grid :: "nat ⇒ real" where "grid m = 2 * pi * real m / real M"

definition steer :: "(nat ⇒ real × real) ⇒ real ⇒ nat ⇒ complex" where
  "steer p φ j = cis (kw * (fst (p j) * cos φ + snd (p j) * sin φ))"

definition patt :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ real ⇒ complex" where
  "patt p N c φ = (∑j<N. steer p φ j * c j)"

definition sq :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ nat ⇒ real" where
  "sq p N c m = (cmod (patt p N c (grid m)))^2"

definition c0 :: "(nat ⇒ real × real) ⇒ real ⇒ nat ⇒ complex" where
  "c0 p t j = cnj (steer p t j)"

definition corr :: "(nat ⇒ real × real) ⇒ nat ⇒ nat ⇒ (nat ⇒ complex) ⇒ nat ⇒ complex"
    ↪ where
  "corr p N s c j = c j - patt p N c (grid s) / of_nat N * cnj (steer p (grid s) j)"
```

LISTING 1. The pattern (`Perturbative_Basics.thy`).

2.2. **One correction.**

Definition 2.1 (correction)**.** For a grid index s, the correction at s is $C_s(w) = w - \frac{F_w(\varphi_s)}{N}\,\bar{a}(\varphi_s)$, where the bar denotes complex conjugation.

Lemma 2.2 (a correction places a null)**.** $F_{C_s(w)}(\varphi_s) = 0$ *for every* w *and* s.

Proof. $F_{C_s(w)}(\varphi_s) = F_w(\varphi_s) - \frac{F_w(\varphi_s)}{N}\sum_j a_j(\varphi_s)\bar{a}_j(\varphi_s) = F_w(\varphi_s) - \frac{F_w(\varphi_s)}{N}\,N = 0$, because $|a_j(\varphi_s)| = 1$. □

```
lemma patt_corr_self: "patt p N (corr p N s c) (grid s) = 0"
```

LISTING 2. Lemma 2.2 in Isabelle (proof omitted).

Remark 2.3 (the correction is the smallest one)**.** With the inner product $\langle u, v\rangle = \sum_j u_j\bar{v}_j$ we have $F_w(\varphi_s) = \langle w, \bar{a}(\varphi_s)\rangle$ and $\|\bar{a}(\varphi_s)\|^2 = N$, so C_s is the orthogonal projection of w onto the hyperplane $\{v : F_v(\varphi_s) = 0\}$. It is therefore the smallest change of the weights that puts a null at φ_s, as in the classical null synthesis of Steyskal [18], and $\|C_s(w)\|^2 = \|w\|^2 - |F_w(\varphi_s)|^2/N$: corrections never increase the norm of the weights. This remark is not used in the proofs.

The next lemma is the key to everything that follows: the effect of any fixed sequence of corrections on the pattern is linear in the weights it started from, with coefficients that do not depend on those weights.

Lemma 2.4 (rows)**.** *Let* $g(m, s) = \frac{1}{N}\sum_j a_j(\varphi_m)\bar{a}_j(\varphi_s)$*. For a sequence of corrections* σ *define the* rows $R^{\sigma}_{m,j}$ *by* $R^{()}_{m,j} = a_j(\varphi_m)$ *and* $R^{(\sigma,s)}_{m,j} = R^{\sigma}_{m,j} - g(m, s)R^{\sigma}_{s,j}$*, where* (σ, s) *is* σ *followed by a correction at* s*. If* C_σ *applies the corrections of* σ *in order, then*

$$F_{C_\sigma(w)}(\varphi_m) = \sum_j R^{\sigma}_{m,j}\,w_j \qquad \text{for every } w \text{ and } m.$$

Algorithm 1 The perturbative method (one beam, one correction per iteration).

$w \leftarrow \bar{a}(t)$ ▷ unit amplitudes; the phases steer the beam to t
while $E(w) \neq \emptyset$ **do**
  $w \leftarrow C_{s(w)}(w)$ ▷ null the loudest eligible sidelobe
end while
return w

Proof. By induction on σ, using $F_{C_{s(v)}}(\varphi_m) = F_v(\varphi_m) - g(m,s)\,F_v(\varphi_s)$, which follows from the definitions by expanding the sum. □

In Isabelle a sequence of corrections is a list with the most recent correction first, and `exc p N c` σ applies the corrections σ to the weights `c` (Listing 3).

```
primrec rows :: "(nat ⇒ real × real) ⇒ nat ⇒ nat list ⇒ nat ⇒ nat ⇒ complex" where
  "rows p N [] m j = steer p (grid m) j"
| "rows p N (s # σ) m j = rows p N σ m j - gram p N m s * rows p N σ s j"

primrec exc :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ nat list ⇒ nat ⇒ complex"
    ↪ where
  "exc p N c [] = c"
| "exc p N c (s # σ) = corr p N s (exc p N c σ)"
```

Listing 3. Rows and Lemma 2.4.

```
lemma patt_exc_rows: "patt p N (exc p N c σ) (grid m) = (∑j<N. rows p N σ m j * c j)"
```

2.3. **The loop.** The method has the following inputs: the array; a focus angle $t \in \mathbb{R}$; a guard band, given by a half-width $\Delta \in (0,\pi)$ or absent; and two *tie rules* τ_m and τ_s, functions that choose an element from every nonempty finite set of grid indices. Let $d(\alpha,\beta) = |\alpha - \beta - 2\pi\,\mathrm{round}((\alpha-\beta)/2\pi)| \in [0,\pi]$ be the distance of two angles on the circle. For weights w define:

- the *strict peaks*: $\mathrm{Pk}(w) = \{m : P_w(m) > P_w(m-1) \text{ and } P_w(m) > P_w(m+1)\}$;
- the *main lobe*: $m_0(w) = \tau_m\big(\arg\min_{m\in\mathrm{Pk}(w)} d(\varphi_m, t)\big)$, the strict peak nearest to the focus;
- *outside the band*: $d(\varphi_m, t) > \Delta$ (true for every m if there is no band);
- the *eligible sidelobes*: $E(w)$, the set of $m \in \mathrm{Pk}(w)$ with $m \neq m_0(w)$, $P_w(m) > \max_q P_w(q)/100$, and m outside the band;
- the *selected sidelobe*: $s(w) = \tau_s\big(\arg\max_{m\in E(w)} P_w(m)\big)$.

The condition $P_w(m) > \max_q P_w(q)/100$ says that the level $10\log_{10}(P_w(m)/\max_q P_w(q))$ is above -20 dB.

Algorithm 1 is the method. It has no iteration cap. Its *run* is the sequence $w_0 = \bar{a}(t)$ and $w_{n+1} = w_n$ if $E(w_n) = \emptyset$, $w_{n+1} = C_{s(w_n)}(w_n)$ otherwise. The run *stops within D corrections* if $E(w_n) = \emptyset$ for some $n \leq D$. Algorithm 1 is the MATLAB code `perturb_method_v25b.m` of the project, restricted to one beam, no prescribed null and one correction per iteration, with its strict-peak rule (`findpeaks_fast`) and in exact arithmetic. Figure 2 shows a run.

The formal definitions are in Listing 4. Two names need explanation. `in_band band t m` means "m is in the sidelobe region", that is, outside the guard band; `band` is `None` for no band and `Some` Δ for the half-width Δ. `tie_rule tm ts` says only that each of the two functions picks an element of every nonempty finite set; nothing else is assumed about them.

```
definition sqmax :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ real" where
```

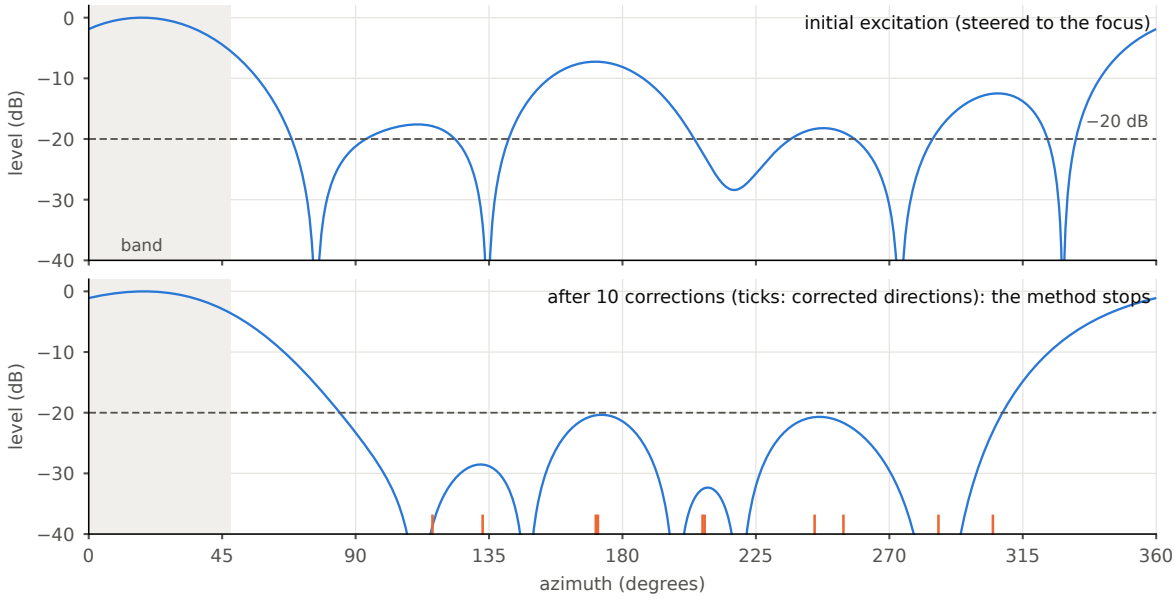


FIGURE 2. A run of Algorithm 1 for the double cross, focus 17.9°, guard band 30° (shaded). Top: the level $10\log_{10}(P_w(m)/\max_q P_w(q))$ of the initial pattern; the loudest sidelobe outside the band is at −7.3 dB. Bottom: after 10 corrections, at the directions marked at the bottom, every strict peak outside the band is below −20 dB and the method stops. (Floating-point simulation; Isabelle's interval evaluation of the same run gives the same ten corrections.)

```
  "sqmax p N c = Max ((sq p N c) ‘ {..<M})"

definition peaks :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ nat set" where
  "peaks p N c = {m. m < M ∧ is_peak p N c m}"

definition mains :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ real ⇒ nat set" where
  "mains p N c t = {m ∈ peaks p N c. ∀q ∈ peaks p N c. cdist (grid m) t ≤ cdist (grid q) t}"

definition in_band :: "real option ⇒ real ⇒ nat ⇒ bool" where
  "in_band band t m ⟷ (case band of None ⇒ True | Some Δ ⇒ cdist (grid m) t > Δ)"

definition eligible :: "(nat ⇒ real × real) ⇒ nat ⇒ real option ⇒ (nat ⇒ complex) ⇒ real
    ↪ ⇒ nat ⇒ nat set" where
  "eligible p N band c t m0 = {m ∈ peaks p N c. m ≠ m0 ∧ sq p N c m > sqmax p N c / 100 ∧
    ↪ in_band band t m}"

definition sel :: "(nat ⇒ real × real) ⇒ nat ⇒ (nat ⇒ complex) ⇒ nat set ⇒ nat set" where
  "sel p N c E = {s ∈ E. ∀m ∈ E. sq p N c m ≤ sq p N c s}"

definition tie_rule :: "(nat set ⇒ nat) ⇒ (nat set ⇒ nat) ⇒ bool" where
  "tie_rule tm ts ⟷ (∀S. S ≠ {} ∧ finite S ⟶ tm S ∈ S) ∧ (∀S. S ≠ {} ∧ finite S ⟶ ts
    ↪ S ∈ S)"

definition elig :: "(nat ⇒ real × real) ⇒ nat ⇒ real option ⇒ (nat set ⇒ nat) ⇒ (nat ⇒
    ↪ complex) ⇒ real ⇒ nat set" where
```

```
  "elig p N band tm c t = eligible p N band c t (tm (mains p N c t))"

primrec run :: "(nat ⇒ real × real) ⇒ nat ⇒ real option ⇒ (nat set ⇒ nat) ⇒ (nat set ⇒
    ↪ nat) ⇒ real ⇒ nat ⇒ nat ⇒ complex" where
  "run p N band tm ts t 0 = c0 p t"
| "run p N band tm ts t (Suc n) =
     (if elig p N band tm (run p N band tm ts t n) t = {} then run p N band tm ts t n
      else corr p N (ts (sel p N (run p N band tm ts t n) (elig p N band tm (run p N band tm
    ↪ ts t n) t))) (run p N band tm ts t n))"

definition stops_within :: "(nat ⇒ real × real) ⇒ nat ⇒ real option ⇒ (nat set ⇒ nat) ⇒
    ↪ (nat set ⇒ nat) ⇒ real ⇒ nat ⇒ bool" where
  "stops_within p N band tm ts t D ⟷ (∃n ≤ D. elig p N band tm (run p N band tm ts t n) t =
    ↪ {})"
```

Listing 4. The method (`Algorithm.thy`).

3. The stopping condition

3.1. **What the stopping test says.** When the loop exits, $E(w) = \emptyset$, which unfolds to the following.

Lemma 3.1. *If $E(w) = \emptyset$, then every strict peak $m \neq m_0(w)$ outside the band satisfies $10 \log_{10}(P_w(m)/\max_q P_w(q)) \leq -20$.*

This is the test that the code implements; call it (I). The specification that one usually has in mind is stronger:

(S) every grid direction outside the band satisfies $P_w(m) \leq \max_q P_w(q)/100$.

(S) implies (I), but not conversely: (I) looks only at strict peaks. A sidelobe whose peak lies inside the band can have a skirt that extends outside the band above -20 dB without being a strict peak there, and (I) does not see it. Theorem 3 is about (S); Theorems 1 and 4 are about (I).

3.2. **The parts of the condition.** Each part of the stopping test matters for whether and how the loop stops.

The threshold is relative to the current maximum. Every correction changes the main lobe as well as the sidelobes, so the threshold $\max_q P_w(q)/100$ moves from one iteration to the next.

The guard band. The main lobe has a width. Without a band, every strict peak except the one nearest the focus counts as a sidelobe, including peaks on the flank of a broad main lobe.

The grid. The test looks at the pattern only at the $M = 1000$ grid directions, and there only at strict peaks. The focus is not restricted to the grid: the theorems hold for every real focus angle. Section 3.4 describes what the test does not see.

Ties. Two peaks can be at the same distance from the focus, or have the same power. For symmetric arrays such exact ties occur, for instance when the focus lies on a symmetry axis. The code breaks ties by taking the first index, as MATLAB's `min` and `sort` do; our theorems hold for every tie rule, so they cover this rule and any other.

3.3. **Why the loop need not stop.** The norm of the weights decreases at every correction (Remark 2.3), but this does not force termination, because the stopping test is relative: the pattern can shrink as a whole while keeping a sidelobe above the threshold. Two phenomena show that the stopping condition deserves attention.

The condition can be unsatisfiable. For the circular array, Theorem 3 shows that no weights at all satisfy (S) for a 20° or 30° band.

The loop can stop for a bad reason. The test can also be met by destroying the beam.

Proposition 3.2 (a degenerate stop)**.** *Suppose that all elements lie on the x-axis, that the focus $t = \varphi_m$ is a grid direction with $m \notin \{0, M/2\}$ that is a strict peak of the initial pattern, that $d(-t, t) > \Delta$, and that the tie rule selects the mirror index $M - m$ in the first iteration. Then $w_1 = 0$, and the run stops after one correction with the zero pattern.*

Proof. Since $y_j = 0$, $a_j(-\varphi) = a_j(\varphi)$ for all φ. Hence $P_{w_0}(M - m) = P_{w_0}(m) = |\sum_j a_j(t)\bar{a}_j(t)|^2 = N^2$, the largest possible value, and $M - m$ is a strict peak because the grid is symmetric under $m \mapsto M - m$. The main lobe is m (distance 0 from the focus), and $M - m$ lies outside the band, so $M - m$ is eligible and as loud as any peak. If it is selected, then $w_1 = \bar{a}(t) - \frac{N}{N}\bar{a}(-t) = 0$. The zero pattern has no strict peaks, so $E(w_1) = \emptyset$. □

So "the method stops" and "the method succeeds" are different statements. Our theorems are about the first, together with the sidelobe property of Lemma 3.1 at the stop; they say nothing about the level of the main lobe at the end. (Proposition 3.2 is not formalized.)

3.4. **What the test does not see.** Two angles occur in the theorems. The focus t is arbitrary: Theorems 1 and 2 hold for every real t, because each interval of foci in the proof is covered completely (Proposition 5.2). The look direction φ is not: the method evaluates the pattern only at the grid directions φ_m, and its test, and hence Lemma 3.1, concerns only those M values and only their strict peaks. At the stop, the pattern can be above −20 dB in three ways that the test does not see.

The numbers below come from a floating-point simulation with the 30° band (without a band the runs are the same), at 4,501 equally spaced focus angles in $[0, \pi/4]$ and at the focus angles where the sequence of corrections changes, located by bisection. At each stop the pattern was evaluated on a grid 64 times finer. They are observations, not theorems.

The main lobe is wider than the band. Both arrays are about one wavelength across. Just outside the 30° band the stopped pattern is between −4.5 and −2.3 dB for the double cross, and between −4.0 and −2.7 dB for the octagon; this is the flank of the main lobe. So (S) fails at every stop that we simulated, and for these arrays only the implemented test (I) can be met.

Flat tops. A sidelobe whose top falls on two adjacent grid points with equal values has no strict peak, so the test does not see it. This does happen. As the focus moves, the top of a sidelobe moves from one grid point to the next; on the way the difference of the two values changes sign, and by the intermediate value theorem there is a focus at which it is zero. If that sidelobe is the only one still above −20 dB, the run stops there without correcting it. We found 82 such focus angles for the double cross, where the missed sidelobe is between −20.0 and −5.2 dB, and 15 for the octagon, between −19.5 and −15.2 dB. For the double cross near $t = 0.312°$ the run stops after no correction at all, with a sidelobe at −5.2 dB. These focus angles are isolated, but the theorems cover them, and at them the method stops early.

Between grid points. The test compares grid values with the largest grid value. When a sidelobe ends exactly at the threshold, its true maximum between two grid points can lie slightly above −20 dB. For the double cross we found 8 such stops, with an excess of at most 0.0004 dB; for the octagon we found none.

None of this affects Theorems 1 and 2, which are about stopping. It limits what the stop means.

3.5. **Exact and floating-point arithmetic.** Our theorems concern the method in exact real arithmetic. A floating-point implementation can also stop for reasons that have nothing to do

with the pattern. As the weights shrink, the computed powers eventually underflow, the levels in decibels become $-\infty$ or `NaN`, and the comparisons that define $E(w)$ become false. In a floating-point simulation of the circular array of Theorem 3 without a guard band, the implemented code ran for 35,414 corrections and then stopped in this way, with a sidelobe still at about -3 dB. This is an observation, not a theorem. For the double cross the runs stop after at most 20 corrections, and in our simulations nothing of this kind happens, but we have not proved a floating-point version of Theorem 1.

3.6. **The hypotheses of our theorems.** For definiteness, everything we prove is about Algorithm 1 with: exact arithmetic; the grid $M = 1000$; the threshold -20 dB relative to the current maximum on the grid; strict peaks; the main lobe defined as the strict peak nearest the focus; one correction per iteration at a loudest eligible peak; arbitrary tie rules; initial weights $\bar{a}(t)$; and element positions equal to the binary64 values given in Appendix A.

4. Results

Theorem 1 (the double cross stops)**.** *Let p be the double cross (Appendix A), $N = 9$. For every focus angle $t \in \mathbb{R}$, for the guard band $\Delta = 30^\circ$ and for no band, and for all tie rules τ_m, τ_s, the run of Algorithm 1 stops within 20 corrections.*

```
theorem double_cross_stops_band30:
  "tie_rule tm ts ⟹ stops_within dcp 9 (Some (pi / 6)) tm ts t 20"
  by (rule dc_pieces_terminates [OF band30_in [where p = 53, folded dc_b30_def] all_c30])

theorem double_cross_stops_no_band:
  "tie_rule tm ts ⟹ stops_within dcp 9 None tm ts t 20"
  by (rule dc_pieces_terminates [OF band_none all_nc])
```

Listing 5. Theorem 1 (`Double_Cross_Final.thy`). `dcp` is the double cross; $\pi/6 = 30^\circ$.

The bound 20 is the depth of the deepest branch that the proof explores, over all focus angles and all tie rules. For one tie rule (the first index, as in MATLAB) and in floating point, the runs use at most 16 corrections (Fig. 3); the bound of the theorem also covers the other tie rules, whose runs can be longer.

The same holds for a second nine-element array, the regular octagon: a centre element and eight elements at distance 48.8 mm, at the angles $k\pi/4$. So that the symmetry argument of Section 5.6 applies exactly, its positions are the binary64 numbers $(\pm a, 0)$, $(0, \pm a)$ and $(\pm b, \pm b)$, with a the binary64 number nearest 48.8 mm and b the one nearest $a/\sqrt{2}$ (Appendix A); the diagonal elements are then within 10^{-17} m of the circle.

Theorem 2 (the octagon stops)**.** *Let p be the octagon (Appendix A), $N = 9$. For every focus angle $t \in \mathbb{R}$, for the guard band $\Delta = 30^\circ$ and for no band, and for all tie rules τ_m, τ_s, the run of Algorithm 1 stops within 10 corrections.*

```
theorem octagon_stops_band30:
  "tie_rule tm ts ⟹ stops_within ocp 9 (Some (pi / 6)) tm ts t 10"
  by (rule oc_pieces_terminates [OF band30_in [where p = 53, folded oc_b30_def] all_c30])

theorem octagon_stops_no_band:
  "tie_rule tm ts ⟹ stops_within ocp 9 None tm ts t 10"
  by (rule oc_pieces_terminates [OF band_none all_nc])
```

Listing 6. Theorem 2 (`Octagon_Final.thy`). `ocp` is the octagon.

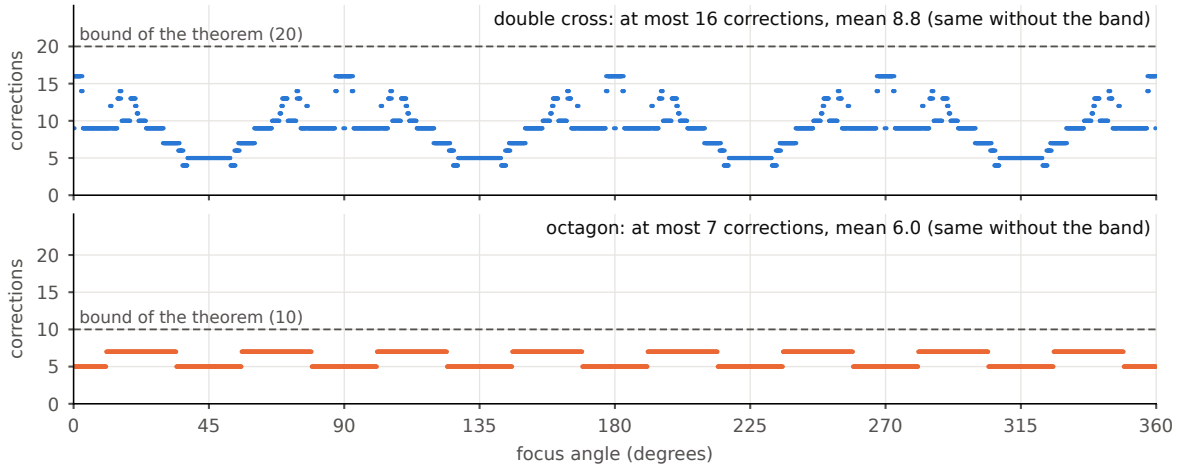


FIGURE 3. Corrections used at 3,600 focus angles by the double cross (top) and the octagon (bottom), for one tie rule (first index) in a floating-point simulation, with the 30° band; without the band the numbers are the same at every angle. Theorems 1 and 2 bound them by 20 and 10 for every focus angle and every tie rule.

Theorem 3 (the circular array cannot meet (S))**.** *Let p consist of 32 elements equally spaced on the circle of radius* 0.4 *$m = 4\lambda$, let the focus be* 0*, and let $Q_\Delta = \{m : d(\varphi_m, 0) > \Delta\}$ be the grid directions outside the band. For every weight vector $w \in \mathbb{C}^{32}$,*

$$\max_{m \in Q_{20^\circ}} |F_w(\varphi_m)| \;\geq\; 0.1614 \max_m |F_w(\varphi_m)|, \qquad \max_{m \in Q_{30^\circ}} |F_w(\varphi_m)| \;\geq\; 0.1286 \max_m |F_w(\varphi_m)|.$$

Consequently, if $w \neq 0$, then (S) fails for both bands: since $20 \log_{10} 0.1614 \approx -15.8$ and $20 \log_{10} 0.1286 \approx -17.8$, some direction outside the band is above -20 dB.

```
definition maxQ :: "nat ⇒ (nat ⇒ complex) ⇒ real" where
  "maxQ Δ c = Max ((λq. cmod (Fc c q)) ‘ Qset Δ)"
definition maxAll :: "(nat ⇒ complex) ⇒ real" where
  "maxAll c = Max ((λm. cmod (Fc c m)) ‘ {..<M})"
definition l2 :: "(nat ⇒ complex) ⇒ real" where
  "l2 c = L2_set (λj. cmod (c j)) {..<32}"
```

LISTING 7. Theorem 3 (`Circular_Floor.thy`, `Circular_Result.thy`). Angles in `Qset` are in degrees; `cv` is an arbitrary weight vector.

```
theorem circular_floor_20: "1614 / 10000 * maxAll cv ≤ maxQ 20 cv"
  by (rule floor_from_check [OF check20 _ K20]) simp_all

theorem circular_floor_30: "1286 / 10000 * maxAll cv ≤ maxQ 30 cv"
  by (rule floor_from_check [OF check30 _ K30]) simp_all


theorem circular_cannot_stop_20: "∃j<32. cv j ≠ 0 ⟹ ¬ maxQ 20 cv ≤ 1/10 * maxAll cv"
  by (rule no_stop_from_check [OF check20 _ K20]) simp_all

theorem circular_cannot_stop_30: "∃j<32. cv j ≠ 0 ⟹ ¬ maxQ 30 cv ≤ 1/10 * maxAll cv"
  by (rule no_stop_from_check [OF check30 _ K30]) simp_all
```

Theorem 4 (total correctness of the program)**.** *Write Algorithm 1 as a while-program over the state of Section 7. If the run with focus t, band Δ and tie rules τ_m, τ_s stops within D corrections, then the program halts from every initial state; at the end, $n \le D$, the weights equal w_n, and every strict peak other than the main lobe and outside the band is at or below -20 dB. In particular, this holds with $D = 20$ for the double cross and with $D = 10$ for the octagon, for every focus, both bands and all tie rules.*

The formal statement is in Listings 18 and 21 of Section 7.

5. Proof of Theorem 1

5.1. **Strategy.** The proof has four steps.

(1) By the symmetry of the double cross, it is enough to treat focus angles in $[0, \pi/4]$ (Section 5.6).
(2) Cover $[0, \pi/4]$ by the 3,232 intervals $[j\,2^{-12}, (j+1)\,2^{-12}]$, $j < 3232$; they reach $3232 \cdot 2^{-12} \approx 0.7891 > \pi/4 \approx 0.7854$.
(3) For each interval T, a checker explores every sequence of corrections that a run with focus in T could make, splitting T where necessary. Success certifies that every run with focus in T stops within 20 corrections, for every tie rule (Sections 5.2 to 5.5).
(4) The checker succeeds on all $2 \times 3232 = 6464$ intervals (one set for each band). Each success is a lemma proved inside Isabelle by evaluating the checker (Section 5.7).

5.2. **Runs as paths in a tree.** Fix a sequence of corrections σ. By Lemma 2.4, after the corrections σ the power at grid index m, as a function of the focus, is

$$P_m(t) = S(R^{\sigma}_m; t), \qquad S(r;t) = \Big|\sum_j r_j\, e^{-\mathrm{i}\psi_j(t)}\Big|^2, \qquad \psi_j(t) = k(x_j \cos t + y_j \sin t), \tag{2}$$

because the initial weights are $\bar{a}_j(t) = e^{-\mathrm{i}\psi_j(t)}$. The rows R^{σ} do not depend on t. Therefore the state of a run is determined by its focus t and by the sequence σ of corrections it has made. All runs with focus in an interval T are paths in one tree: its nodes are correction sequences, and the children of σ are (σ, s) for the indices s that a run in the state σ could correct next. In Isabelle this is `sq_exc_SS` together with `run_exc`, which says that the run is the initial weights corrected by the list `corrs` of the corrections made so far.

```
lemma sq_exc_SS: "sq p N (exc p N (c0 p t) σ) m = SS (rows p N σ m) p N t"
```

```
lemma run_exc: "run p N band tm ts t n = exc p N (c0 p t) (corrs p N band tm ts t n)"
  by (induction n) simp_all
```

5.3. **Enclosures on an interval of focus angles.** On an interval $T = [c-h, c+h]$ we bound $S(r;\cdot)$ by its Taylor expansion at the midpoint.

Lemma 5.1 (second-order bound)**.** *For $r \in \mathbb{C}^N$ let $B = \sum_j |r_j|$, $D = \sum_j |r_j|\gamma_j$ and $E = \sum_j |r_j|(\gamma_j + \gamma_j^2)$, with $\gamma_j = k\,\|p_j\|$, and let $H = 2(D^2 + BE)$. Then for all $t, c \in \mathbb{R}$,*

$$|S(r;t) - S(r;c)| \le |S'(r;c)|\,|t-c| + \tfrac{H}{2}(t-c)^2.$$

Proof. Write $G(t) = \sum_j r_j e^{-\mathrm{i}\psi_j(t)}$, so that $S = |G|^2$ and $S'' = 2|G'|^2 + 2\,\mathrm{Re}(\bar{G}G'')$. Since $|\psi_j'| \le \gamma_j$ and $|\psi_j''| \le \gamma_j$, we have $|G| \le B$, $|G'| \le D$ and $|G''| \le E$, hence $|S''| \le H$. Taylor's theorem with the Lagrange remainder gives the bound. □

```
theorem SS_taylor:
  "|SS r p N t - SS r p N c| ≤ |SS1 r p N c| * |t - c| + Hc r p N / 2 * (t - c)^2"
```

LISTING 8. Lemma 5.1 (`Taylor_Enclosure.thy`). `SS r p N t` is $S(r;t)$, `SS1` its derivative, `Hc` the constant H; `gam p j` is γ_j.

```
definition Bc :: "(nat ⇒ complex) ⇒ nat ⇒ real" where "Bc r N = (∑j<N. cmod (r j))"
definition Dc :: "(nat ⇒ complex) ⇒ (nat ⇒ real × real) ⇒ nat ⇒ real" where
  "Dc r p N = (∑j<N. cmod (r j) * gam p j)"
definition Ec :: "(nat ⇒ complex) ⇒ (nat ⇒ real × real) ⇒ nat ⇒ real" where
  "Ec r p N = (∑j<N. cmod (r j) * (gam p j + (gam p j)^2))"
definition Hc :: "(nat ⇒ complex) ⇒ (nat ⇒ real × real) ⇒ nat ⇒ real" where
  "Hc r p N = 2 * ((Dc r p N)^2 + Bc r N * Ec r p N)"
```

A second bound of the same kind, with constant $H(r)+H(r')$, controls the difference $S(r;t)-S(r';t)$ of two powers directly. This is sharper than subtracting two separate enclosures, and it is what decides the peak tests $P_m > P_{m\pm1}$.

The checker evaluates $S(r;c)$ and $S'(r;c)$ at the midpoint with interval arithmetic on binary floating-point numbers with outward rounding and 53-bit precision; cosines and sines are enclosed by the verified approximations of Isabelle's `Approximation` library [11]. The rows themselves are carried along the tree as interval enclosures: every correction updates them by the recursion of Lemma 2.4, evaluated in interval arithmetic. Lemma 5.1 then gives, for every $t \in T$, an interval containing $P_m(t)$, and one containing $P_m(t) - P_{m\pm1}(t)$.

5.4. **Classifying the grid directions at a node.** At a node (T, σ) the checker uses these enclosures to classify every grid index m, in a way that is valid for every focus $t \in T$ simultaneously.

- m is *possibly a peak* (`PP`) if both neighbour differences may be positive, and *certainly a peak* (`CP`) if both are certainly positive.
- m is *possibly* or *certainly outside the band* (`PO`, `CO`), from bounds on $d(\varphi_m, t)$ over T; the distance is 1-Lipschitz in t.
- m is *excluded* (`EXC`) if it lies in a run of one or two adjacent possible peaks that is nearer to every focus in T than every other possible peak. If such a run contains a strict peak, that peak is the main lobe; since two adjacent grid points cannot both be strict peaks, nothing in the run can be eligible.
- The lower bounds give $\mathrm{GL} = \max_m \underline{P_m} \le \max_q P(q)$ and the threshold $\mathrm{TH} = \mathrm{GL}/100$, rounded downward. The upper bounds give US, the largest upper bound $\overline{P_m}$ over the indices that could be eligible (possible peaks, possibly outside the band, not excluded).

The node is *terminal* if $\mathrm{US} \le \mathrm{TH}$. Then no index is eligible, for any focus in T (Listing 9, `terminal_sound`). Otherwise the *candidates* are the indices that could be eligible and whose upper bound reaches TH, except those whose upper bound is below LL. Here LL is the largest lower bound of an index that is certainly a strict peak, certainly outside the band and certainly not the main lobe; such an index is louder than any index whose upper bound is below LL, and eligible whenever that index is, so the latter cannot be selected. Every index that some run could select is a candidate (`sel_cand`).

```
definition PP where "PP p q N R Hb lo hi m ⟷ m < M ∧ 0 < upper (DP p q N R Hb lo hi m) ∧ 0
    ↪ < upper (DN p q N R Hb lo hi m)"
definition CP where "CP p q N R Hb lo hi m ⟷ m < M ∧ 0 < lower (DP p q N R Hb lo hi m) ∧ 0
    ↪ < lower (DN p q N R Hb lo hi m)"
```

```
definition PO where "PO p lo hi n0 band m ⟷ (case band of None ⇒ True | Some D ⇒ lower D <
    ↪ DMAX p lo hi n0 m)"
definition CO where "CO p lo hi n0 band m ⟷ (case band of None ⇒ True | Some D ⇒ upper D <
    ↪ DMIN p lo hi n0 m)"
definition PM where "PM p q N R Hb lo hi n0 m ⟷ PP p q N R Hb lo hi m ∧
   (∀k < M. CP p q N R Hb lo hi k ⟶ DMIN p lo hi n0 m ≤ DMAX p lo hi n0 k)"

definition run_ok where "run_ok p q N R Hb lo hi n0 RS ⟷
   (∀r ∈ RS. PP p q N R Hb lo hi r) ∧
   (∀r ∈ RS. ∀k < M. PP p q N R Hb lo hi k ∧ k ∉ RS ⟶ DMAX p lo hi n0 r < DMIN p lo hi n0
    ↪ k)"
definition EXC where "EXC p q N R Hb lo hi n0 m ⟷ m < M ∧
   (run_ok p q N R Hb lo hi n0 {m} ∨ run_ok p q N R Hb lo hi n0 {m, next_idx m} ∨ run_ok p q
    ↪ N R Hb lo hi n0 {prev_idx m, m})"

definition GL where "GL p q N R Hb lo hi = Max ((LOB p q N R Hb lo hi) ‘ {..<M})"
definition US where "US p q N R Hb lo hi n0 band =
   Max (insert 0 ((UPB p q N R Hb lo hi) ‘ {m. PP p q N R Hb lo hi m ∧ PO p lo hi n0 band m ∧
    ↪ ¬ EXC p q N R Hb lo hi n0 m}))"

definition inv100_ivl :: "float interval" where
  "inv100_ivl = Ivl (Float 11529215046068469 (-60)) (Float 11529215046068470 (-60))"

lemma inv100_in: "1 / 100 ∈ᵣ inv100_ivl"
  unfolding inv100_ivl_def by (intro in_real_intervalI) (simp_all add: Ivl_simps
    ↪ real_of_float_eq)

definition TH where "TH p q N R Hb lo hi = mult_float_interval p (interval_of (GL p q N R Hb
    ↪ lo hi)) inv100_ivl"

definition terminal where "terminal p q N R Hb lo hi n0 band ⟷ US p q N R Hb lo hi n0 band ≤
    ↪  lower (TH p q N R Hb lo hi)"

definition LL where "LL p q N R Hb lo hi n0 band =
   Max (insert 0 ((LOB p q N R Hb lo hi) ‘ {k. CP p q N R Hb lo hi k ∧ CO p lo hi n0 band k ∧
    ↪ ¬ PM p q N R Hb lo hi n0 k}))"

definition cand where "cand p q N R Hb lo hi n0 band m ⟷
   PP p q N R Hb lo hi m ∧ PO p lo hi n0 band m ∧ ¬ EXC p q N R Hb lo hi n0 m ∧
   lower (TH p q N R Hb lo hi) ≤ UPB p q N R Hb lo hi m ∧
   ¬ (0 < LL p q N R Hb lo hi n0 band ∧ UPB p q N R Hb lo hi m < LL p q N R Hb lo hi n0 band)"
```

LISTING 9. Classification of a node and its two soundness lemmas (`Classify.thy`). `R` and `Hb` are the row enclosures and curvature bounds of the node; `lo`, `hi` are the ends of T.

```
lemma terminal_sound:
  assumes m0: "m0 ∈ mains (fpos q) N c t" and trm: "terminal p q N R Hb lo hi n0 band"
  shows "eligible (fpos q) N bd c t m0 = {}"

lemma sel_cand:
  assumes m0: "m0 ∈ mains (fpos q) N c t" and s: "s ∈ sel (fpos q) N c (eligible (fpos q) N
    ↪ bd c t m0)"
```

```
  shows "cand p q N R Hb lo hi n0 band s"
```

5.5. **The search and its soundness.** The search (Listing 10) starts at the root $\sigma = ()$ over an interval T. At a node it first checks the distance bounds (`all_ok`); if the node is terminal it succeeds. Otherwise it computes the candidates. If T is wider than $w_{\min} = 2^{-26}$, and either the number of candidates is not exactly one or the depth limit $D_{\max} = 20$ has been reached, it splits T at its midpoint `tc lo hi` and searches both halves. If it does not split, it succeeds when there is no candidate, because a selected index would be a candidate; it fails when the depth limit has been reached; and otherwise it searches every child (σ, s), one for each candidate s. The argument `f` is fuel, a bound on the recursion depth that makes `verify` a total function.

```
fun verify :: "nat ⇒ (nat ⇒ float × float) ⇒ nat ⇒ float interval ⇒ (float ⇒ float ⇒
    ↪ nat ⇒ int) ⇒
              float interval option ⇒ nat ⇒ float ⇒ nat ⇒ float ⇒ float ⇒ nat list ⇒
    ↪ bool" where
  "verify p q N IN n0f band Dmax wmin 0 lo hi σ = False"
| "verify p q N IN n0f band Dmax wmin (Suc f) lo hi σ =
     (let R = Rref p q N IN σ; Hb = Hbref p q N IN σ; n0 = n0f lo hi in
      if ¬ all_ok p lo hi n0 then False
      else if terminal p q N R Hb lo hi n0 band then True
      else let K = cands p q N R Hb lo hi n0 band in
        if (length K ≠ 1 ∨ Dmax ≤ length σ) ∧ wmin < hi - lo
          then verify p q N IN n0f band Dmax wmin f lo (tc lo hi) σ ∧ verify p q N IN n0f band
    ↪ Dmax wmin f (tc lo hi) hi σ
        else if K = [] then True
        else if Dmax ≤ length σ then False
        else list_all (λs. verify p q N IN n0f band Dmax wmin f lo hi (s # σ)) K)"
```

Listing 10. The search (`Search.thy`).

Proposition 5.2 (soundness)**.** *If the search succeeds on* $[\mathrm{lo}, \mathrm{hi}]$*, then for every* $t \in [\mathrm{lo}, \mathrm{hi}]$ *and all tie rules the run stops within* $D_{\max}$ *corrections.*

Proof. We prove, by induction on the fuel, that success at a node $([\mathrm{lo}, \mathrm{hi}], \sigma)$ implies: every run whose focus lies in $[\mathrm{lo}, \mathrm{hi}]$ and whose first corrections are σ stops within $D_{\max}$. At a terminal node the run's state is described by the node (Section 5.2), and `terminal_sound` gives $E = \emptyset$. At a split, the focus lies in one of the halves. At a branching node, the run either has stopped, or its next correction is a candidate by `sel_cand`, and the child for that candidate certifies the rest of the run. Exact ties need no special treatment: the candidates include every index that any tie rule could select. The root has $\sigma = ()$, which every run starts with. □

The executable checker `check_sector` (Listing 11) computes the kernel and the curvature tables once and runs an implementation of the search on lists; `verify_impl_eq` proves it equal to `verify`.

```
theorem verify_sound:
  assumes IN: "1 / real N ∈ᵣ IN" and bd: "band_in bd band" and tr: "tie_rule tm ts"
  shows "verify p q N IN n0f band Dmax wmin f lo hi σ ⟹ length σ ≤ Dmax ⟹
         real_of_float lo ≤ t ⟹ t ≤ real_of_float hi ⟹
         corrs (fpos q) N bd tm ts t (length σ) = σ ⟹
         stops_within (fpos q) N bd tm ts t Dmax"

theorem check_sector_sound:
```

```
  assumes IN: "1 / real N ∈ᵣ IN" and bd: "band_in bd band" and tr: "tie_rule tm ts"
    and chk: "check_sector p q N IN band Dmax wmin f lo hi"
    and t: "real_of_float lo ≤ t" "t ≤ real_of_float hi"
  shows "stops_within (fpos q) N bd tm ts t Dmax"
```

LISTING 11. The executable entry point and Proposition 5.2 (`Exec_Search.thy`). `q` gives the positions as binary floats and `fpos q` as reals; `IN` encloses $1/N$.

5.6. **Symmetry.** The double cross is invariant under the rotation by $\pi/2$ and under the reflection in the line $y = x$; both permute its elements. On the grid, the rotation is $m \mapsto m + 250$ and the reflection is $m \mapsto 250 - m$ (indices mod 1000). Both maps preserve neighbourhoods (the reflection exchanges $m - 1$ and $m + 1$), and they carry distances from the focus t to distances from $t + \pi/2$ and from $\pi/2 - t$. They therefore carry the run with focus t and tie rules (τ_m, τ_s) to the run with the transformed focus and the transported tie rules, with the same number of corrections (`all_rules_transport`). Every real angle is carried into $[0, \pi/4]$ by these maps and the 2π-periodicity of the steering vector (`dc_stops_all`).

This is one reason why Theorem 1 is stated for all tie rules: the symmetry changes the tie rule, so a statement about one fixed rule would not transfer.

```
theorem all_rules_transport:
  assumes "⋀tm ts. tie_rule tm ts ⟹ stops_within p N bd tm ts t D"
  shows "tie_rule tm' ts' ⟹ stops_within p N bd tm' ts' (G t) D"
```

LISTING 12. The reduction of every focus angle to the first eighth of the circle (`Symmetry.thy`, `Double_Cross_Symmetry.thy`).

```
definition dc_stops :: "real option ⇒ real ⇒ nat ⇒ bool" where
  "dc_stops bd t D ⟷ (∀tm ts. tie_rule tm ts ⟶ stops_within dcp 9 bd tm ts t D)"
```

```
theorem dc_stops_all:
  assumes base: "⋀t. 0 ≤ t ⟹ t ≤ pi/4 ⟹ dc_stops bd t D"
  shows "dc_stops bd t D"
```

5.7. **The computation.** Each of the 3,232 intervals, for each band, is one lemma. It states that the checker succeeds and is proved by evaluating the compiled checker (Listing 13). The method `dc_eval` is Isabelle's `computation_check` [7] with logging. The 6,464 lemmas are grouped into 24 sessions, and a final theory assembles them with the symmetry argument into Theorem 1 (Listing 5).

```
definition dc_sub :: "float interval option ⇒ nat ⇒ bool" where
  "dc_sub band j = check_sector 53 dc_q 9 inv9_ivl band 20 (Float 1 (-26)) 100 (Float (int j)
    ↪ (-12)) (Float (int j + 1) (-12))"
```

LISTING 13. One interval and its proof (generated theory `DCX_c30_000.thy`).

```
lemma dc_c30_0000: "dc_sub dc_b30 0"
  by (dc_eval dc_c30_0000)
```

Table 2 gives the size of the computation. To make it fast we replaced the library's code equations for the power-of-two operations on floats (division and remainder by 2^k, bit length, alignment of exponents in addition) by shift-based equations. Each replacement is proved equal to the original, so it adds nothing to the trusted base. An independent C++ implementation

	double cross	octagon
intervals per band ($[j\,2^{-12}, (j+1)\,2^{-12}]$)	3,232	3,232
lemmas (two bands)	6,464	6,464
depth bound $D_{\max}$	20	10
smallest width still split, $w_{\min}$	2^{-26} rad	2^{-26} rad
search nodes (C++ implementation, each band)	162,467	80,681
wall time in Isabelle2026, 10 threads	10 h 15 min	5 h 17 min
generated piece theories	26,500 lines	26,400 lines

TABLE 2. The computations behind Theorems 1 and 2. The method, the enclosures, the checker, its soundness and the symmetry argument take about 3,100 lines of theory text; the octagon reuses them and adds a 300-line copy of the symmetry theory.

of the same checker, which is not trusted and serves only to choose the parameters, explores 162,467 nodes and reports the same depth bound 20 for both bands.

The octagon. The proof of Theorem 2 is the same, with the octagon's positions and the depth bound 10. The octagon is invariant under the same rotation and reflection as the double cross, and since its elements are listed in the same order the symmetry argument carries over unchanged; it is a copy of the one for the double cross with the positions replaced (`Octagon_Symmetry.thy`). The C++ implementation explores 80,681 nodes for the octagon, about half as many as for the double cross.

6. Proof of Theorem 3

Let $Q = Q_\Delta$ and write $\|F_w\|_Q = \max_{q\in Q} |F_w(\varphi_q)|$. The proof bounds the pattern inside the band by the pattern outside it, for every weight vector at once.

Step 1: certificates. Suppose that for a grid direction $\psi \notin Q$ there are coefficients $\mu_q \in \mathbb{C}$, $q \in Q$, with $\sum_q |\mu_q| \le \Lambda$ and $\big\|a(\varphi_\psi) - \sum_q \mu_q\, a(\varphi_q)\big\|_2 \le e$. Then, for every w,

$$|F_w(\varphi_\psi)| = \Big|\sum_q \mu_q F_w(\varphi_q) + \big(a(\varphi_\psi) - \textstyle\sum_q \mu_q a(\varphi_q)\big)^{\top} w\Big| \le \Lambda\, \|F_w\|_Q + e\, \|w\|_2$$

by the triangle and Cauchy–Schwarz inequalities.

Step 2: the sidelobes control the weights. Write w in real form as a vector in $\mathbb{R}^{64}$; then $\sum_{q\in Q} |F_w(\varphi_q)|^2 = w^{\top} G w$ for an explicit real Gram matrix G. If $G = LL^{\top} + s^2 I + E$ for some matrix L, a number s, and a remainder E whose row and column sums of absolute values are at most δ, then $w^{\top} G w \ge (s^2 - \delta)\|w\|^2$, so $\|w\|_2^2 \le |Q|\, \|F_w\|_Q^2/(s^2-\delta)$. The matrix L comes from a Cholesky factorization computed outside Isabelle; Isabelle only checks the bound on E.

Step 3. Combining the two steps, $|F_w(\varphi_\psi)| \le K\, \|F_w\|_Q$ for every $\psi \notin Q$, with $K = \Lambda + e\sqrt{|Q|/(s^2-\delta)}$; for $\psi \in Q$ the same holds trivially as long as $K \ge 1$. Hence $\max_m |F_w(\varphi_m)| \le K\, \|F_w\|_Q$. The certificates, one for each grid direction inside the band, were computed by linear programming and checked in Isabelle by evaluation. They give $0.1614\,K \le 1$ for $\Delta = 20^\circ$ and $0.1286\,K \le 1$ for $\Delta = 30^\circ$ ($K \approx 6.1958$ and $K \approx 7.7760$), which is Theorem 3. □

```
definition certificate :: "nat ⇒ nat ⇒ (nat ⇒ complex) ⇒ real ⇒ real ⇒ bool" where
  "certificate Δ ψ μ Λ e ⟷ (∑q∈Qset Δ. cmod (μ q)) ≤ Λ ∧
     (∑j<32. (cmod (steer ucp (grid ψ) j - (∑q∈Qset Δ. μ q * steer ucp (grid q) j)))²) ≤
    ↪ e²"
```

LISTING 14. Certificates and the general floor theorem (`Circular_Floor.thy`).

```
theorem floor_from_certificates:
  fixes Lm :: "nat ⇒ nat ⇒ real" and s2 δ Λ e K :: real and Δ :: nat
  defines "E ≡ λi k. Gm Δ i k - LLt Lm i k - (if i = k then s2 else 0)"
  assumes Qne: "Qset Δ ≠ {}"
    and row: "⋀i. i < 64 ⟹ (∑k<64. |E i k|) ≤ δ" and col: "⋀k. k < 64 ⟹ (∑i<64. |E i
    ↪ k|) ≤ δ"
    and pos: "0 < s2 - δ" and e0: "0 ≤ e" and LK: "Λ ≤ K" and K1: "1 ≤ K"
    and KK: "e² * real (card (Qset Δ)) ≤ (K - Λ)² * (s2 - δ)"
    and cert: "⋀ψ. ψ < M ⟹ ψ ∉ Qset Δ ⟹ ∃μ. certificate Δ ψ μ Λ e"
  shows "maxAll c ≤ K * maxQ Δ c"
```

The theorem is a statement about all weights, not only about those the method reaches; it is a static obstruction. For a 45° band the corresponding linear program gives a floor of about −20.6 dB, below the threshold, so this argument says nothing for that band.

7. The program and its total correctness

7.1. **Programs and triples.** Isabelle/UTP provides a while-language whose programs denote interaction trees [22, 4]. The state is a record of typed variables (`zstore`), and `H[P] S [Q]` is a total-correctness triple [10, 3]: from every state satisfying `P`, the program `S` terminates in a state satisfying `Q`. Loops carry an invariant and a variant, a natural number that decreases at every iteration, and the method `vcg` reduces a triple to verification conditions in plain higher-order logic [2].

7.2. **The program.** The state (Listing 15) holds the weights `w`, the number of corrections `n`, the powers `P` and their maximum `Pmax`, a grid index `k`, the set of strict peaks `pk`, the distance `dmin` and the set `Mn` of the peaks nearest the focus, the main lobe `m0`, the eligible set `E`, the loudest eligible power `best` and the indices `Ls` that attain it, and the corrected index `ms`.

```
zstore pstate =
  w :: "nat ⇒ complex"
  n :: nat
  P :: "nat ⇒ real"
  Pmax :: real
  k :: nat
  pk :: "nat set"
  dmin :: real
  Mn :: "nat set"
  m0 :: nat
  E :: "nat set"
  best :: real
  Ls :: "nat set"
  ms :: nat
```

Listing 15. The state.

The program `scan` (Listing 16) computes everything that the loop test needs, by five passes over the grid: the powers and their maximum, the strict peaks, the distance of the nearest peak, the nearest peaks and the main lobe, and the eligible set. Each pass is a loop with an invariant that states what it has computed so far; `below R k` is the set of indices $q < k$ with `R q`. An annotation `x = old:x` says that the loop does not change `x`: `old:x` is the value of `x` on entry to the loop.

```
program scan "(p :: nat ⇒ real × real, N :: nat, band :: real option, tm :: nat set ⇒ nat, t
    ↪ :: real)"
```

```
  over pstate =
"k := 0; Pmax := 0;
 while k ≠ M
   invariant w = old:w ∧ n = old:n ∧ k ≤ M ∧ agrees P (sq p N w) k ∧
     Pmax = max_below 0 (sq p N w) (λq. True) k
   variant M - k
 do P := fupd P k ((cmod (patt p N w (grid k)))^2); Pmax := max Pmax (P k); k := k + 1 od;
 k := 0; pk := {};
 while k ≠ M
   invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ Pmax = old:Pmax ∧ k ≤ M ∧
     pk = below (is_peak p N w) k
   variant M - k
 do if P (prev_idx k) < P k ∧ P (next_idx k) < P k then pk := insert k pk fi; k := k + 1 od;
 k := 0; dmin := 2 * pi;
 while k ≠ M
   invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ Pmax = old:Pmax ∧ pk = old:pk ∧ k ≤ M ∧
     dmin = min_below (2 * pi) (λq. cdist (grid q) t) (λq. q ∈ pk) k
   variant M - k
 do if k ∈ pk then dmin := min dmin (cdist (grid k) t) fi; k := k + 1 od;
 k := 0; Mn := {};
 while k ≠ M
   invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ Pmax = old:Pmax ∧ pk = old:pk ∧
     dmin = old:dmin ∧ k ≤ M ∧ Mn = below (λq. q ∈ pk ∧ cdist (grid q) t = dmin) k
   variant M - k
 do if k ∈ pk ∧ cdist (grid k) t = dmin then Mn := insert k Mn fi; k := k + 1 od;
 m0 := tm Mn;
 k := 0; E := {};
 while k ≠ M
   invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ Pmax = old:Pmax ∧ pk = old:pk ∧
     m0 = old:m0 ∧ k ≤ M ∧ E = below (λq. q ∈ elig p N band tm w t) k
   variant M - k
 do if k ∈ pk ∧ k ≠ m0 ∧ 10 * log 10 (P k / Pmax) > -20 ∧ in_band band t k then E := insert
    ↪ k E fi;
   k := k + 1 od"
```

LISTING 16. The scan (`Perturbative_UTP.thy`).

The method itself is Listing 17: scan, then, while some sidelobe is eligible, find the loudest eligible power, the indices that attain it, choose one with the tie rule, correct, count, and scan again. `perturb_ann` is the same program with the main loop annotated by an invariant and the variant `D - n`. The bound `D` occurs only in the annotation, and the two programs are equal (`perturb_ann_is_perturb`), because annotations do not change what a program does. In particular, `D` is not an iteration cap: the program `perturb` has none.

```
program perturb_ann "(p :: nat ⇒ real × real, N :: nat, band :: real option, tm :: nat set ⇒
    ↪ nat,
    ts :: nat set ⇒ nat, t :: real, D :: nat)" over pstate =
"w := (λj. cnj (steer p t j)); n := 0;
 scan (p, N, band, tm, t);
 while E ≠ {}
   invariant w = Algorithm.run p N band tm ts t n ∧ n ≤ D ∧ (∀q<M. P q = sq p N w q) ∧
     m0 = tm (mains p N w t) ∧ E = elig p N band tm w t
   variant D - n
 do
```

```
  k := 0; best := 0;
  while k ≠ M
    invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ E = old:E ∧ m0 = old:m0 ∧ k ≤ M ∧
      best = max_below 0 (sq p N w) (λq. q ∈ E) k
    variant M - k
  do if k ∈ E then best := max best (P k) fi; k := k + 1 od;
  k := 0; Ls := {};
  while k ≠ M
    invariant w = old:w ∧ n = old:n ∧ P = old:P ∧ E = old:E ∧ m0 = old:m0 ∧ best = old:best
   ↪ ∧
      k ≤ M ∧ Ls = below (λq. q ∈ E ∧ sq p N w q = best) k
    variant M - k
  do if k ∈ E ∧ P k = best then Ls := insert k Ls fi; k := k + 1 od;
  ms := ts Ls;
  w := (λj. w j - patt p N w (grid ms) / of_nat N * cnj (steer p (grid ms) j));
  n := n + 1;
  scan (p, N, band, tm, t)
od"
```

Listing 17. The method, with its main loop annotated.

```
lemma perturb_ann_is_perturb: "perturb_ann (p, N, band, tm, ts, t, D) = perturb (p, N, band,
   ↪ tm, ts, t)"
  by (simp add: perturb_ann_def perturb_def while_inv_var_def)
```

7.3. **The triple and its proof.** Listing 18 is Theorem 4 for an arbitrary array. The hypothesis says that the run of Section 2.3 stops within `D` corrections. The variant `D - n` decreases because a run that has not stopped after `n` corrections has `n < D`: if `n` were at least the step at which the run stops, the run would already be frozen there and nothing would be eligible (`busy_less`, Listing 19). The rest of the invariant ties the program's variables to the mathematical run: `w` is the state after `n` corrections, the powers are those of `w`, and `m0` and `E` are its main lobe and eligible set. When the loop exits, `E` is empty and Lemma 3.1 gives the postcondition.

```
theorem perturb_ann_correct:
  assumes stop: "stops_within p N band tm ts t D"
  shows "H[True] perturb_ann (p, N, band, tm, ts, t, D)
    [w = Algorithm.run p N band tm ts t n ∧ n ≤ D ∧ m0 = tm (mains p N w t) ∧
     (∀m<M. is_peak p N w m ∧ m ≠ m0 ∧ in_band band t m ⟶
        10 * log 10 (sq p N w m / sqmax p N w) ≤ -20)]"
  unfolding perturb_ann_def scan_def
  apply (simp only: prod.case kcomp_assoc)
  apply vcg
  apply (simp_all only: fun_eq_iff [symmetric] c0_lam corr_lam sq_patt)
  apply (simp_all add: scan_simps agrees_fupd fupd_at busy_less [OF stop])
  apply (auto simp: c0_def corr_def dest: busy_mem [OF stop] intro: stopped_log)
  apply (auto simp: below_def is_peak_def elig_mem less_Suc_eq)
  done
```

Listing 18. Theorem 4 for an arbitrary array (`Perturbative_UTP.thy`).

```
lemma busy_less:
  assumes "stops_within p N band tm ts t D"
    and "elig p N band tm (run p N band tm ts t n) t ≠ {}"
```

```
  shows "n < D"
```

Listing 19. Why the variant decreases (`Perturbative_Scans.thy`).

```
corollary perturb_correct:
  assumes "stops_within p N band tm ts t D"
  shows "H[True] perturb (p, N, band, tm, ts, t)
    [w = Algorithm.run p N band tm ts t n ∧ n ≤ D ∧ m0 = tm (mains p N w t) ∧
     (∀m<M. is_peak p N w m ∧ m ≠ m0 ∧ in_band band t m ⟶
        10 * log 10 (sq p N w m / sqmax p N w) ≤ -20)]"
  using perturb_ann_correct [OF assms] by (simp only: perturb_ann_is_perturb)
```

After the program definitions are unfolded, `vcg` produces 111 verification conditions, and four calls of the simplifier and of `auto` discharge them with the lemmas that describe the scans. One proof rule had to be added (Listing 20): it lets the VCG pass a loop that is followed by more code, using the precondition that the prestate rule of the library carries through the loop. Without it the VCG stops at every loop that is followed by more code, and an intermediate assertion has to be supplied by hand. The theorem uses no oracle: Isabelle's oracle check returns the empty set.

```
lemma thl_while_seq [hoare_safe]:
  assumes
    "⋀ old z. H[P⟦«old»/v⟧ ∧ @(Inv old) ∧ G ∧ Var = «z»] Body [@(Inv old) ∧ Var < «z»]"
    "⋀ old. ‘P ∧ «old» = $v ⟶ @(Inv old)‘"
    "⋀ old. H[¬ G ∧ P⟦«old»/v⟧ ∧ @(Inv old)] Rest [Post]"
  shows "H[P] (while G inv @(Inv old) var Var do Body od) ;; Rest [Post]"
```

Listing 20. The rule for a loop followed by more code.

Listing 21 instantiates the triple for the double cross with Theorem 1. The no-band case is the same with `None` in place of `Some (pi / 6)`, and the octagon instance (`Octagon_UTP.thy`) is the same with `ocp`, Theorem 2 and the bound 10.

```
theorem double_cross_band30:
  assumes "tie_rule tm ts"
  shows "H[True] perturb (dcp, 9, Some (pi / 6), tm, ts, t)
    [w = Algorithm.run dcp 9 (Some (pi / 6)) tm ts t n ∧ n ≤ 20 ∧ m0 = tm (mains dcp 9 w t) ∧
     (∀m<M. is_peak dcp 9 w m ∧ m ≠ m0 ∧ in_band (Some (pi / 6)) t m ⟶
        10 * log 10 (sq dcp 9 w m / sqmax dcp 9 w) ≤ -20)]"
  by (rule perturb_correct [OF double_cross_stops_band30 [OF assms]])
```

Listing 21. Theorem 4 for the double cross (`Double_Cross_UTP.thy`).

8. Related work

Computer-assisted proofs with interval arithmetic. Interval arithmetic [13] turns finitely many floating-point evaluations into rigorous bounds valid on whole intervals [21]. Tucker's proof that the Lorenz attractor exists [19, 20] combines an exact reduction with a rigorous ODE solver and subdivision of a compact set. Hales' proof of the Kepler conjecture [9] reduced the conjecture to finitely many linear programs and nonlinear inequalities checked by computer. Our proof of Theorem 1 has the same shape: exact lemmas (Lemmas 2.2, 2.4 and 5.1, the symmetry) reduce the theorem to finitely many interval statements, one for each interval of focus angles.

Checking the computations formally. Both proofs were later checked in proof assistants. Immler verified an ODE solver in Isabelle/HOL and used it to re-check the computational part

of Tucker's proof [12]. The Flyspeck project produced a formal proof of the Kepler conjecture [8], including verified nonlinear inequalities proved with Taylor interval approximations [17]. Our checker follows this approach: it is defined in higher-order logic, proved sound (Proposition 5.2), and executed by the code generator [7]. Our Taylor bound (Lemma 5.1) plays the role of the Taylor interval approximations of Solovyev and Hales, specialized to the squared modulus of a trigonometric sum. Isabelle's `approximation` method [11] proves single real inequalities in the same way; we use its verified enclosures of the cosine.

Verified numerical methods. In earlier work we verified numerical methods such as bisection, fixed-point iteration and gradient descent as Isabelle/UTP programs with total-correctness triples [2]. Section 7 uses the same framework. The new ingredients here are a program whose termination depends on a computer-assisted theorem, and a proof rule that composes a loop with the code that follows it.

Pattern nulling. The correction C_s is the minimum-norm perturbation that imposes one null [18]. Perturbation methods for nulling remain in use [6]. We know of no earlier termination analysis of the iterative method with a relative stopping threshold.

9. What is not proved

We list the limits of the results.

- Exact arithmetic only. A floating-point version would have to account for rounding in the weights and the powers (Section 3.5).
- The pattern at the stop. The theorems say that the run stops, and that at the stop every strict peak of the sampled pattern outside the band, other than the main lobe, is at or below -20 dB. They say nothing about the pattern between grid points, or about a sidelobe whose top falls on two grid points of equal value. At some focus angles the stopped pattern is above -20 dB in both places (Section 3.4); this is shown by simulation, not proved.
- The positions. The arrays are fixed binary64 positions. Robustness under perturbation of the positions is not proved.
- The circular array. Theorem 3 concerns (S), not the implemented test (I), and only the 20° and 30° bands. Simulations suggest that the implemented method does not stop there either, but that is not proved.
- The main lobe. The theorems do not say how strong the main lobe is when the method stops (Proposition 3.2).
- Other arrays. Each theorem is about one specific array. The method of proof applies to any array, but each new array needs its own computation, and its own symmetry argument if the reduction of Section 5.6 is to be used.
- Untrusted tools. The C++ checker and the linear-programming solver are not trusted; they only produce parameters and certificates that Isabelle checks.

10. Conclusion

Whether the perturbative method stops depends on the array. For the double cross it stops after at most 20 corrections and for the regular octagon after at most 10, for every focus angle and every tie rule; for a large circular array its stopping condition cannot be met at all. Stopping is not the same as success: the test sees only the sampled pattern and its strict peaks, and at isolated focus angles the method stops with a sidelobe above -20 dB that the test misses (Section 3.4). The positive result needed a computer-assisted proof, and we obtained one whose computational part is itself a theorem of Isabelle/HOL. The total-correctness triple then turns the theorem about runs into a theorem about the program, with no iteration cap.

Appendix A. The positions

The double cross has its centre element at the origin, four elements at $(\pm a, 0)$ and $(0, \pm a)$, and four at $(\pm b, \pm b)$, with $a = 3041648138404103 \cdot 2^{-56}$ m ≈ 42.211 mm and $b = 5645503481367791 \cdot 2^{-57}$ m ≈ 39.174 mm, so that the diagonal elements are at distance $b\sqrt{2} \approx 55.40$ mm from the centre. These are the binary64 values of the project's beamforming code.

```
definition double_cross_pos :: "(float × float) list" where
  "double_cross_pos = [(Float 0 0, Float 0 0), (Float (-3041648138404103) (-56), Float 0 0),
    ↪ (Float 3041648138404103 (-56), Float 0 0), (Float 0 0, Float 3041648138404103 (-56)),
    ↪ (Float 0 0, Float (-3041648138404103) (-56)), (Float 5645503481367791 (-57), Float
    ↪ 5645503481367791 (-57)), (Float (-5645503481367791) (-57), Float 5645503481367791
    ↪ (-57)), (Float (-5645503481367791) (-57), Float (-5645503481367791) (-57)), (Float
    ↪ 5645503481367791 (-57), Float (-5645503481367791) (-57))]"
```

Listing 22. The positions of the double cross, in metres (`Double_Cross.thy`).

The octagon uses the same element order, with $a = 7032821178101767 \cdot 2^{-57}$ m, the binary64 number nearest 48.8 mm, on the axes, and $b = 1243238886477031 \cdot 2^{-55}$ m ≈ 34.507 mm, the binary64 number nearest $a/\sqrt{2}$, on the diagonals.

```
definition octagon_d4_pos :: "(float × float) list" where
  "octagon_d4_pos = [(Float 0 0, Float 0 0), (Float (-7032821178101767) (-57), Float 0 0),
    (Float 7032821178101767 (-57), Float 0 0), (Float 0 0, Float 7032821178101767 (-57)),
    (Float 0 0, Float (-7032821178101767) (-57)), (Float 1243238886477031 (-55), Float
    ↪ 1243238886477031 (-55)),
    (Float (-1243238886477031) (-55), Float 1243238886477031 (-55)),
    (Float (-1243238886477031) (-55), Float (-1243238886477031) (-55)),
    (Float 1243238886477031 (-55), Float (-1243238886477031) (-55))]"
```

Listing 23. The positions of the octagon, in metres (`Octagon_Symmetry.thy`).

Appendix B. Reproducing the proofs

All theories build with Isabelle2026. In the directory `Proof/Isabelle` of the project:

- `isabelle build -b -d . -d Pieces -j 1 -o threads=10 DCX_Final` builds the method, the checker, its soundness proof, the circular array and the 6,464 pieces, ending with Theorem 1 (about 10 hours);
- `isabelle build -b -d . -d Pieces -d Pieces_Oct -j 1 -o threads=10 OCX_Final` does the same for the octagon, ending with Theorem 2 (about 5 hours);
- `tools/build_utp.sh Double_Cross_UTP` and `tools/build_utp.sh Octagon_UTP` build the programs and Theorem 4 on top of them (about 7 minutes each); they need the ITree/UTP libraries ported to Isabelle2026, and the heaps that the option `-b` above keeps.

References

1. Constantine A. Balanis, *Antenna theory: Analysis and design*, 4 ed., John Wiley & Sons, Hoboken, NJ, 2016.
2. Dustin Bryant, Jonathan Julián Huerta y Munive, and Simon Foster, *Verifying numerical methods with Isabelle/HOL*, Software Engineering and Formal Methods (SEFM 2026), Lecture Notes in Computer Science, Springer, 2026, To appear.
3. Robert W. Floyd, *Assigning meanings to programs*, Mathematical Aspects of Computer Science, Proc. Sympos. Appl. Math., vol. 19, Amer. Math. Soc., Providence, RI, 1967, pp. 19–32.
4. Simon Foster, Chung-Kil Hur, and Jim Woodcock, *Unifying model execution and deductive verification with Interaction Trees in Isabelle/HOL*, ACM Trans. Softw. Eng. Methodol. (2024).

5. Simon Foster, Frank Zeyda, Yakoub Nemouchi, Pedro Ribeiro, and Burkhart Wolff, *Isabelle/UTP: Mechanised theory engineering for unifying theories of programming*, Archive of Formal Proofs (2019).
6. Yuanzhe Gong, Arish Yaseen, Tingrui Zhang, Robert Morawski, and Tho Le-Ngoc, *Perturbation-based nulling control beamforming with measured element radiation patterns for MU-mMIMO*, IEEE Open J. Veh. Technol. **5** (2024), 1273–1293.
7. Florian Haftmann and Tobias Nipkow, *Code generation via higher-order rewrite systems*, Functional and Logic Programming (FLOPS 2010), Lecture Notes in Computer Science, vol. 6009, Springer, 2010, pp. 103–117.
8. Thomas Hales et al., *A formal proof of the Kepler conjecture*, Forum Math. Pi **5** (2017), e2.
9. Thomas C. Hales, *A proof of the Kepler conjecture*, Ann. of Math. (2) **162** (2005), no. 3, 1065–1185.
10. C. A. R. Hoare, *An axiomatic basis for computer programming*, Commun. ACM **12** (1969), no. 10, 576–580.
11. Johannes Hölzl, *Proving inequalities over reals with computation in Isabelle/HOL*, Programming Languages for Mechanized Mathematics Systems (PLMMS), ACM, 2009, pp. 38–45.
12. Fabian Immler, *A verified ODE solver and the Lorenz attractor*, J. Autom. Reason. **61** (2018), no. 1-4, 73–111.
13. Ramon E. Moore, *Interval analysis*, Prentice-Hall, Englewood Cliffs, NJ, 1966.
14. Mosier, *[the perturbative sidelobe-suppression method: full reference to be supplied]*, Citation to be completed.
15. Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel, *Isabelle/HOL: A proof assistant for higher-order logic*, Lecture Notes in Computer Science, vol. 2283, Springer, Berlin, 2002.
16. Lawrence C. Paulson, *Isabelle: The next 700 theorem provers*, Logic and Computer Science (P. Odifreddi, ed.), Academic Press, London, 1990, pp. 361–386.
17. Alexey Solovyev and Thomas C. Hales, *Formal verification of nonlinear inequalities with Taylor interval approximations*, NASA Formal Methods (NFM 2013), Lecture Notes in Computer Science, vol. 7871, Springer, 2013, pp. 383–397.
18. Hans Steyskal, *Synthesis of antenna patterns with prescribed nulls*, IEEE Trans. Antennas Propag. **30** (1982), no. 2, 273–279.
19. Warwick Tucker, *The Lorenz attractor exists*, C. R. Acad. Sci. Paris Sér. I Math. **328** (1999), no. 12, 1197–1202.
20. ———, *A rigorous ODE solver and Smale's 14th problem*, Found. Comput. Math. **2** (2002), no. 1, 53–117.
21. ———, *Validated numerics: A short introduction to rigorous computations*, Princeton University Press, Princeton, NJ, 2011.
22. Li yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic, *Interaction trees: representing recursive and impure programs in Coq*, Proc. ACM Program. Lang. **4** (2020), no. POPL, 51:1–51:32.

Independent researcher, USA
Email address: brydustin@gmail.com